UniMatrícula: Diseño e Implementación de un Sistema Web de Matrícula Universitaria con ASP.NET Core MVC

Adrian Jose Arias Vargas
Escuela de Ingeniería en Sistemas de Computación
Universidad Fidélitas
San José, Costa Rica
aarias20433@ufide.ac.cr

Abstract*— *El presente artículo documenta el diseño, desarrollo e implementación de UniMatrícula, un sistema web de matrícula universitaria construido sobre el patrón Modelo-Vista-Controlador (MVC) utilizando el framework ASP.NET Core. El sistema permite a estudiantes registrarse, seleccionar una carrera, consultar cursos disponibles y matricularse o cancelar matrículas mediante solicitudes asíncronas (AJAX), mientras que el personal administrador gestiona carreras, cursos, docentes y estudiantes a través de operaciones CRUD completas. Se describe la arquitectura del sistema, el modelo de datos, los mecanismos de seguridad basados en ASP.NET Core Identity con autorización por roles, las funcionalidades extendidas de paginación y filtrado, y las pruebas realizadas para validar el correcto funcionamiento de cada módulo.

Palabras clave*— *ASP.NET Core MVC, matrícula universitaria, Entity Framework Core, ASP.NET Identity, AJAX, Razor, C#.

I. Introducción

La gestión de matrícula universitaria es un proceso administrativo y académico crítico que involucra múltiples actores: estudiantes, docentes y personal administrativo. Los sistemas manuales o basados en hojas de cálculo son propensos a errores de duplicidad, sobrecupo de cursos y falta de trazabilidad histórica. El presente proyecto propone UniMatrícula, una aplicación web desarrollada con ASP.NET Core MVC que centraliza y automatiza el proceso de matrícula, aplicando buenas prácticas de seguridad, separación de responsabilidades y experiencia de usuario dinámica.

El objetivo general del sistema es permitir la gestión eficiente de estudiantes, carreras, cursos y docentes, e implementar un flujo de autoservicio en el cual el propio estudiante matricula o cancela cursos de su carrera, respetando la disponibilidad de cupos, sin intervención directa del administrador para esa operación puntual.

II. Arquitectura y Tecnologías Utilizadas

El sistema se desarrolló siguiendo el patrón arquitectónico Modelo-Vista-Controlador (MVC) provisto de forma nativa por el framework ASP.NET Core [1], el cual separa la lógica de negocio (Modelos), la lógica de presentación (Vistas Razor) y el flujo de control de las solicitudes HTTP (Controladores). Esta separación facilita el mantenimiento y la escalabilidad del sistema.

Para la persistencia de datos se utilizó Entity Framework Core como Object-Relational Mapper (ORM) [2], contra una base de datos SQL Server. El acceso a datos se centraliza en la clase ApplicationDbContext, la cual expone los conjuntos de entidades (DbSet) de Carrera, Curso, Docente, Estudiante y Matricula.

La autenticación y autorización se implementaron con ASP.NET Core Identity [3], que provee gestión de usuarios, contraseñas cifradas y roles. Se definieron dos roles: Administrador y Estudiante, cada uno con acceso restringido a distintas acciones de los controladores mediante el atributo [Authorize].

Para las operaciones de matrícula y cancelación de cursos se implementaron solicitudes asíncronas (AJAX) utilizando la Fetch API nativa del navegador [4], evitando recargas completas de página y mejorando la experiencia de usuario mediante retroalimentación inmediata.

III. Estructura del Sistema

El modelo de datos del sistema está compuesto por cinco entidades principales, además de las entidades propias de ASP.NET Identity (usuario y rol). A continuación se describe cada una y su propósito dentro del sistema.

A. Carrera

Representa un programa académico ofrecido por la universidad (por ejemplo, Ingeniería en Sistemas de Computación). Contiene el código, nombre, descripción, cantidad de semestres y créditos totales. Una carrera se relaciona de forma uno a muchos con Curso y con Estudiante, ya que agrupa los cursos que pertenecen a su plan de estudios y a los estudiantes matriculados en ella.

B. Curso

Representa una materia o asignatura específica de una carrera, con código, nombre, créditos, cupos disponibles y período académico. Cada curso pertenece a una única Carrera (llave foránea CarreraId) y puede tener asignado un Docente

(llave foránea DocenteId, opcional). Un curso puede tener múltiples matrículas asociadas.

C. Docente

Representa al personal académico que imparte los cursos. Contiene nombre completo, correo electrónico, especialidad y un indicador de si se encuentra activo. Un docente puede estar asignado a varios cursos.

D. Estudiante

Representa el perfil académico de un estudiante, vinculado uno a uno con su cuenta de ASP.NET Identity mediante el campo UserId. Contiene nombre completo, cédula (única), correo, fecha de ingreso, estado activo y la carrera a la que pertenece. Un estudiante posee múltiples matrículas a lo largo del tiempo.

E. Matrícula

Es la entidad que vincula a un Estudiante con un Curso en un momento determinado, registrando la fecha de matrícula y su estado (Activa, Cancelada o Completada, modelado mediante una enumeración EstadoMatricula). Esta entidad es el eje central del proceso de autoservicio de matrícula: se crea al matricularse y cambia de estado al cancelarse, preservando el historial en lugar de eliminar el registro.

F. Identidad y Roles

Se utilizan las entidades nativas IdentityUser e IdentityRole de ASP.NET Core Identity para la autenticación. Cada usuario de Identity puede estar asociado a un registro de Estudiante (rol Estudiante) o funcionar como cuenta administrativa (rol Administrador), determinando qué controladores y vistas puede utilizar.

IV. Funcionalidades Implementadas

A. Gestión administrativa (CRUD)

El rol Administrador cuenta con operaciones completas de creación, lectura, actualización y eliminación (CRUD) sobre Carreras, Cursos, Docentes y Estudiantes, incluyendo la asociación de cursos a carreras y la asignación de docentes a cursos desde el propio formulario de curso.

B. Registro y matrícula del estudiante

Un usuario nuevo se registra seleccionando su carrera desde el formulario de registro. Una vez autenticado, el estudiante accede a la vista "Cursos Disponibles", que lista únicamente los cursos de su propia carrera que aún tienen cupo y en los que no está matriculado activamente. Al presionar "Matricularme", una solicitud AJAX valida en el servidor que existan cupos y que no exista una matrícula activa duplicada antes de crear el registro y descontar el cupo, todo sin recargar la página.

Desde "Mi Portal", el estudiante puede visualizar sus cursos matriculados y cancelar una matrícula activa; la cancelación libera el cupo del curso y conserva el registro en un historial, en lugar de eliminarlo.

C. Seguridad

El acceso a los controladores administrativos se restringe con [Authorize(Roles = "Administrador")], mientras que las acciones de matrícula se restringen a [Authorize(Roles = "Estudiante")]. Todas las operaciones de escritura (POST) están protegidas contra ataques de falsificación de solicitudes entre sitios (CSRF) mediante el token antifalsificación de ASP.NET Core, validado tanto en formularios tradicionales como en las solicitudes AJAX.

D. Funcionalidad extendida: paginación y filtros

Para evitar cargar listados completos en memoria y mejorar la usabilidad ante catálogos grandes, se implementó una clase genérica PaginatedList<T> que pagina cualquier consulta LINQ sobre Entity Framework Core. Se aplicó paginación en los listados de Cursos y Carreras, junto con filtros: en Cursos, por texto (código o nombre), carrera, período académico y cantidad de créditos; en Carreras, por texto (código o nombre). Los filtros y la página actual se conservan como parámetros de consulta en la URL, permitiendo compartir o recargar una vista filtrada.

V. Pruebas Realizadas

Se realizaron pruebas funcionales manuales sobre un entorno de desarrollo local (SQL Server + Kestrel), cubriendo los siguientes escenarios: (1) registro de un nuevo estudiante y selección de carrera; (2) inicio de sesión con roles de Estudiante y Administrador; (3) operaciones CRUD de Carreras, Cursos y Docentes; (4) matrícula de un curso disponible, verificando el descuento del cupo y la actualización de la interfaz sin recarga de página; (5) intento de matrícula duplicada, verificando el mensaje de error correspondiente; (6) cancelación de una matrícula activa y verificación de la liberación del cupo; (7) uso de los filtros y la paginación en los listados de Cursos y Carreras; y (8) verificación de que un usuario con rol Estudiante no puede acceder a las rutas administrativas de creación, edición o eliminación.

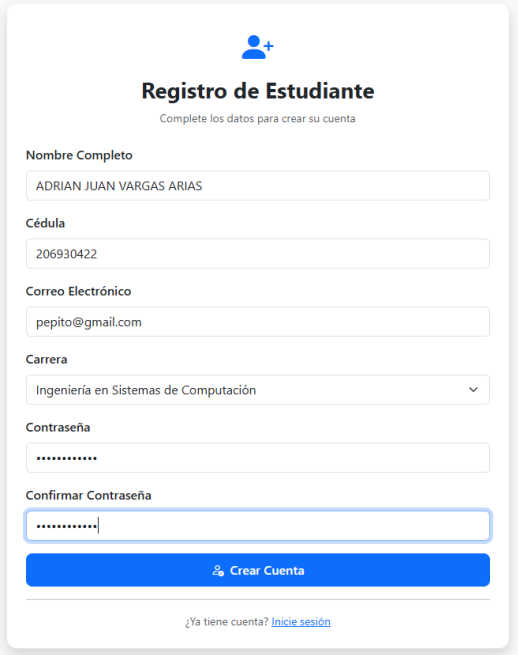


Fig. 1. Registro de un nuevo estudiante seleccionando su carrera.



Fig. 2. Vista de Cursos Disponibles con filtros y matrícula vía AJAX.

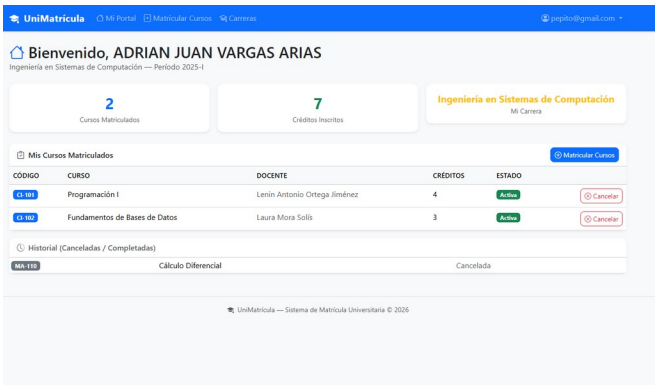


Fig. 3. Portal del Estudiante mostrando cursos matriculados e historial.

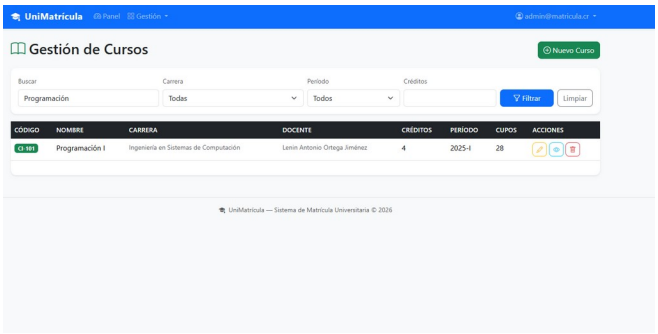


Fig. 4. Panel de Administración con la gestión de Cursos, filtros y paginación.

VI. Manual de Usuario Básico

A. Ingreso al sistema

El usuario nuevo debe registrarse desde la opción "Registrarse", indicando sus datos personales y seleccionando su carrera. Una vez creada la cuenta, inicia sesión desde "Iniciar Sesión" con su correo y contraseña.

B. Uso como Estudiante

- Ingresar a "Mi Portal" para ver un resumen de cursos matriculados y créditos inscritos.
- Ingresar a "Matricular Cursos" para ver los cursos disponibles de su carrera; puede filtrar por período o buscar por nombre/código.
- Presionar "Matricularme" en el curso deseado; el sistema confirma la matrícula de forma inmediata.
- Desde "Mi Portal", presionar "Cancelar" en un curso matriculado para liberar el cupo.

C. Uso como Administrador (Docente/Gestión)

- Ingresar al menú "Gestión" para administrar Carreras, Cursos, Docentes y Estudiantes.
- En "Cursos", usar los filtros de búsqueda, carrera, período o créditos para localizar un curso específico.
- Usar los botones de "Editar", "Detalles" o "Eliminar" según la operación deseada; al crear o editar un curso, se asigna su carrera y, opcionalmente, un docente.

VII. Conclusiones

El desarrollo de UniMatrícula permitió aplicar de forma integral los conceptos del curso de Programación Avanzada sobre ASP.NET Core MVC: separación de responsabilidades mediante el patrón MVC, persistencia de datos con Entity Framework Core, seguridad basada en autenticación y autorización por roles con ASP.NET Identity, e interactividad mediante solicitudes asíncronas AJAX. El resultado es un sistema funcional que resuelve el caso planteado, permitiendo tanto la gestión administrativa completa del catálogo académico como el autoservicio de matrícula por parte del estudiante, con mecanismos que evitan condiciones de sobrecupo y matrícula duplicada.

Como trabajo futuro, el sistema podría extenderse con validación de prerrequisitos entre cursos, generación de comprobantes de matrícula en PDF y notificaciones por correo electrónico ante cambios de estado en una matrícula.

Referencias

[1] Microsoft, "Overview of ASP.NET Core MVC," Microsoft Learn, 2024. [En línea]. Disponible: https://learn.microsoft.com/aspnet/core/mvc/overview

[2] Microsoft, "Overview of Entity Framework Core," Microsoft Learn, 2024. [En línea]. Disponible: https://learn.microsoft.com/ef/core/

[3] Microsoft, "Introduction to Identity on ASP.NET Core," Microsoft Learn, 2024. [En línea]. Disponible: https://learn.microsoft.com/aspnet/core/security/authentication/identity

[4] MDN Web Docs, "Fetch API," Mozilla, 2024. [En línea]. Disponible: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[5] E. Gamma, R. Helm, R. Johnson, y J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA: Addison-Wesley, 1994.